A Monte Carlo Tree Search-Based Mixed Traffic Flow Control Algorithm for Arterial Intersection

Yanqiu Cheng, Ph.D. Student
Highway School, Chang'an University, Xi'an, Shaanxi, China, 710064
Department of Civil, Architectural and Environmental Engineering
Missouri University of Science and Technology, Rolla, MO 65409, USA
Email: chengyan@mst.edu

Xianbiao Hu, Assistant Professor. (Corresponding Author).
Department of Civil, Architectural and Environmental Engineering
Missouri University of Science and Technology, Rolla, MO 65409, USA
Phone:(573) 341-6178
Email: xbhu@mst.edu

Qing Tang, Ph.D. Student
Department of Civil, Architectural and Environmental Engineering
Missouri University of Science and Technology, Rolla, MO 65409, USA
Email: qt7b6@mst.edu

Hongsheng Qi, Associate professor
College of Civil Engineering and Architecture, Zhejiang University
866 Yuhangtang Road, Hangzhou, 310058, China
Email: qihongsheng@zju.edu.cn

Hong Yang, Assistant professor
Department of Computational Modeling and Simulation Engineering
Old Dominion University. 4700 Elkhorn Ave, Norfolk, VA 23529
Email: hyang@odu.edu

Words: 6,519
Tables: 1*250=250
Total Words: 6,769

Submitted for Presentation and Publication at the 99th Annual Meeting of Transportation Research Board

ABSTRACT

This manuscript presented a model-free approach based on Monte Carlo Tree Search (MCTS) algorithm for the control of mixed traffic flow of Human-Driven Vehicles (HDV) and connected and autonomous vehicles (CAV), named MCTS-MTF, on a one-lane roadway with signalized intersection control. Previous researches often simplified the problem with certain assumptions to reduce computational burden, such as dividing a vehicle trajectory into several segments with constant speed or linear acceleration/deceleration which was rather unrealistic. This manuscript departed from the existing research that minimum constraints on CAV trajectory control were required, as long as the basic rules such as safety consideration and vehicular performance limit were followed. Modeling efforts were made to improve the algorithm solution quality and the runtime efficiency over naïve MCTS (n-MCTS) algorithm. This was achieved by an exploration-exploitation balance calibration module, and a tree expansion determination module to more effectively expand the tree along the desired direction. The case study results found that the proposed algorithm was able to achieve a travel time saving of 3.5% and a fuel consumption of 6.5%. It was also demonstrated to run 8 times the speed of a naïve MCTS model, suggesting a promising potential for real-time or near real-time applications.

Keywords: Connected and Autonomous Vehicles (CAV); Trajectory Scheduling; Mixed Traffic Flow; Monte Carlo Tree Search (MCTS); Traffic Flow Modeling;

1. INTRODUCTION

Connected and autonomous vehicle (CAV) technologies suggest great potentials for future transportation systems. Compared with human driven vehicles (HDV), CAVs behave more logically and safely, as they are explicitly controlled by the algorithms, and there is little or no reaction delay in the high computation power of today's processors. In particular, trajectories of CAV can be adjusted with developed intelligence to achieve pre-defined objectives such as minimizing delays and/or fuel consumptions at roadway intersections.

Trajectory scheduling is essentially a constrained non-linear optimization problem, with the objective function being minimizing fuel consumption, emissions or travel time, and the constraints including collision avoidance constraints, speed limit, signal control, vehicle kinematic and comfort constraints and so on. Various models were proposed to solve this problem. For example, Lee and Park developed a cooperative vehicle intersection control algorithm and converted it to a nonlinear constrained optimization problem, so that vehicles can safely cross the intersection without colliding with other vehicles [1]. Yang et al. formulated it as a bi-level optimization problem and proposed a fast branch and bound algorithm to improve the computational efficiency [2]. Similar research can be found in [3] that formulated it as a multi-stage optimization problem, [4] that used model predictive control, [5, 6] that developed a shooting heuristic algorithm and so on.

Another common approach to tackling this non-linear problem is to convert it into a mixed integer linear programming problem. For example, Jin et al.[7] discretized the system states, including speed, acceleration and road grade, and estimated the fuel consumption rate with respect to each discretized system state and constructed a set of look-up tables of step-by-step fuel consumption. Li and Zhou [8] formulated the intersection automation policy optimization as a special case of machine scheduling problem using a mixed integer linear programming formulation, a phase-time-traffic hypernetwork model was developed and sequential branch-and-bound search algorithms was proposed to solve the problem. Research with similar MILP approach can also be found in [9, 10] and so on.

Some research on CAV trajectory scheduling consider signal control plan as given, such as the work in [11], in which a mixed-integer linear programming model was developed to cooperatively optimize the trajectories of CAVs along a corridor. The trajectories of all vehicles were coordinated for system optimality in terms of total vehicle delay along the corridor. A multi-stage optimal control formulation was proposed in [12] to obtain the optimal vehicle trajectory on signalized arterials, where both vehicle queue and traffic light status were considered. A constrained optimization model was proposed as an approximation approach to solve the problem efficiently. Similar work can be found in [13, 14] that considered signal control information as given. On the other hand, some researchers studied how to jointly optimize signal timing plan and CAV planning, such as the work in [15] which proposed a joint traffic signal optimization algorithm based on connected vehicle information to identify optimum signal timing and phasing

plans while also providing speed guidance to individual vehicles to minimize total number of stopping maneuvers. Yu et al. also developed a model to optimize vehicle trajectories and traffic signals in a unified framework at isolated signalized intersections in a CAV environment. Vehicles were split into platoons and their trajectories were determined by optimal control models and car-following models on the basis of optimized arrival times with the objective to minimize fuel consumption and emission [16]. Similar work can also be found in [17, 18] and many others.

Trajectory scheduling is a complex problem as each vehicle trajectory is essentially an infinite-dimensional object [6] and in a mixed traffic flow, the interactions between HDV and CAV, such as the car-following behavior, are complex to model. To solve this problem efficiently, certain assumptions are usually made to simplify the problem solving. One common and effective approach is to divide a vehicle trajectory into several segments with constant acceleration/deceleration to reduce computational burden. For example, He et al. [12] and Wu et al. [3] built approximation models with three trajectory segments, in which vehicles were supposed to accelerate/decelerate to an optimal cruising speed, travel at this speed; and finally accelerate/decelerate to a final speed when passing through the intersection. Therefore, the optimal control problem was transformed into a non-linear optimization problem with much fewer decision variables. Similar approach can also be found in [5, 6, 19], in which each vehicle trajectory was broken into a few sections to make the problem analytically solvable. Such approach decomposed the originally hard trajectory design problem to a simple constructive heuristic, and significantly reduced the computational burden. Such assumptions, however, made the model less realistic and flexible, and essentially sacrificed the modeling realism for runtime efficiency.

A model-free approach based on Monte Carlo Tree Search (MCTS) algorithm for the control of mixed traffic flow of HDV and CAV, named MCTS-MTF was proposed in this manuscript. This manuscript departed from the existing research that minimum constraints on CAV trajectory control were required, as long as the basic rules such as safety consideration and vehicular performance limit was followed. The rationale behind this algorithm design is to treat the CAV trajectory scheduling problem in a similar way as playing computer game GO, in which the CAV randomly selects one move among all possible actions, assesses the potential outcomes, simulate the playout, and use that information to determine the most promising moves and expand the search tree accordingly. Such modeling approach enables a more flexible and realistic vehicle movements scheme, as at any time, a CAV can choose to accelerate, decelerate or cruise at a constant speed. It also allows a two-way interaction so that CAV driving behavior will not only influence, but is also influenced by HDV movements. It should be noted that the “connectivity” in this manuscript is modeled mostly between CAV and infrastructure (V2I), i.e. the traffic signal. The CAV is aware of the traffic signal timing plan in real time, including what light is on and how many seconds are left until the phase changes. The CAV is also aware of its distance to the downstream intersection in real-time. Additionally, the CAV is able to detect the speed and distance headway of the HDV immediate behind it, so that it can be used as input to decide its next move.

Modeling efforts were made to improve the algorithm solution quality and the runtime

efficiency over naïve MCTS (n-MCTS) algorithm. This was achieved by an exploration-exploitation balance calibration module, and a tree expansion determination module to more effectively expand the tree along the desired direction. The proposed algorithm was implemented and tested in a one-lane roadway with signalized intersection control. The results found that compared with the benchmark scenario, the proposed algorithm was able to achieve a travel time saving of 3.5% and a fuel consumption of 6.5%. The proposed algorithm was also demonstrated to run 8 times the speed of a naïve MCTS model, suggesting a promising potential for real-time or near real-time application for CAV trajectory control.

2. NOTATIONS

As a convenient reference, the mathematical notations used in this section are presented below.

t: time index

S^t: The state of this segment at time *t*

Y^t: The set of all vehicles displacement distance from the start position

V^t: The set of all vehicles velocity at time *t*

${D_h}^t$: The set of all vehicles distance headway at time *t*.

A: The set of CAV actions

a: The action of CAV

M: The set of vehicles

l_s: The length of the segment

l_a : The average length of a vehicle

d_s : The length of the signal area

y_i^t: The displacement distance from the start position of i vehicles at time *t*

v_i^t: The speed of i vehicles at time *t*

d_{hi}^t: The distance headway of *i* vehicle at time *t*

t_c: The signal light cycle duration

t_g, t_y, t_r: The green, yellow and red light duration

Δt: The shortest time interval

$n(s)$: The number of times node s has been visited during the search

$n(s,a)$: The number of times action a whenever node s was visited

$Q(s,a)$*:* The averaged result of node s

3. METHODOLOGY

3.1. Naïve MCTS (n-MCTS) Model

The original MCTS is a discrete search framework for sequential decision making and has gained huge success in the computer game Go. Due to its great potential for dealing with large-scale decision problems, it has been used in many fields, such as dynamic resource allocation [20], computer games [21], path planning [22], routing [23] and others. The standard MCTS algorithm (or naïve MCTS, n-MCTS) is revised to account for the domain specific attributes in our study.

Applying the MCTS for solving CAV trajectory scheduling problem, n-MCTS works in a rolling horizontal way. The model assumes that in a mixed traffic flow of CAV and HDV, the trajectory of CAV can be adjusted by the algorithm and the actions of HDV are determined by the surrounding environment, following a car-following model to be presented at a later section. At each decision moment, the model simulates the traffic flow on the one-lane roadway segment within a pre-defined time horizon and progressively updates the CAV movements (including accelerating or decelerating with certain acceleration/deceleration values, or maintain a constant speed), according to given performance indexes (PI). After all simulations are completed, the movement with the best PI is selected. The resulting CAV movement is taken as the input of the roadway segment, and the decision moment then moves forward.

As a heuristic algorithm, MCTS gains knowledge by simulating a given problem and establishes an asymmetric search tree that is guided in the most promising direction. A typical MCTS iteration usually consists of four consecutive steps: selection, expansion, simulation, and back propagation. When the number of iterations is high enough, the results from MCTS will converge to the optimum.

Problem decomposition

In a mixed traffic flow with multiple CAVs and HDVs, the mixed traffic flow queue was firstly decomposed into several “basic interactive unit”, each with a leading CAV and multiple HDV followers. Figure 1 below is used to illustrated the idea.

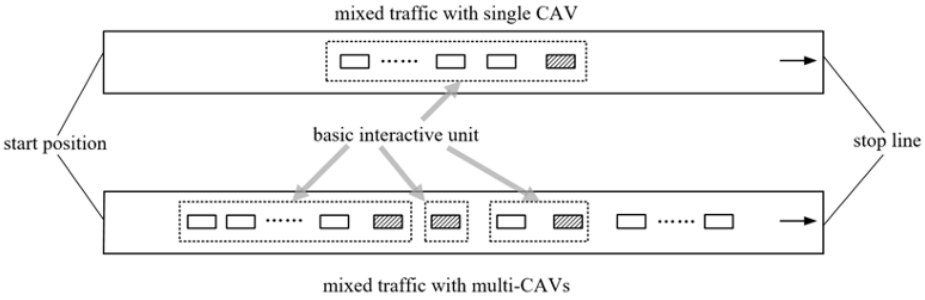


Figure 1. Decomposition of mixed traffic flow into basic interactive units

In the figure above, the mixed traffic flow includes three CAVs and the rest are HDVs. It should be noted that as CAV is connected with infrastructure, the CAV is aware of the traffic signal timing plan in real time, including what light is on and how many seconds are left until the phase changes. The CAV is also aware of its distance to the downstream intersection in real-time. Additionally, the CAV is able to detect the speed and distance headway of the HDV immediate behind it, so that it can be used as input to decide its next move.

Based on the location of CAVs, they can be regrouped into three basic interactive units. The first group has one leading CAV and one following HDV, the second unit has only one CAV, and third unit has one leading CAV and many following HDV. As such, the mixed traffic flow problem can be converted into a trajectory optimization problem for one basic interaction unit, which significantly reduced the computational complexity. The rationales for such decomposition approach are that 1) for the HDVs that are driving ahead of CAV, their trajectory is not subject to the CAV movement behind them, so they can be considered as independent and will drive at free flow speed (FFS); 2) on an arterial roadway segment, following typical traffic flow models all vehicles are assumed to drive at FFS before arriving at intersections. As such, if CAV is driving below FFS, it's slowing down the traffic and enforcing the car following behavior for the HDVs behind it. If the CAV is driving exactly at FFS, its movement is consistent with the HDVs. To summarize, in both scenarios, CAV can be considered as "leading" the HDV platoon behind it, and will naturally form multiple basic interactive units and each unit can be solved with the proposed MCTS-MTF model. Such decomposition is also frequently used in the previous literatures, for example, in [5, 6].

UCT formulation

Kocsis and Szepesvári [24] combined tree search with the selection policy for bandits, and designed one of the most popular MCTS methods – UCT algorithm (upper confidence bounds for trees). The advantage of UCT algorithm is that it can well balance the dilemma between exploration and exploitation part of a selection policy.

The underlying mechanism for UCT, that is denoted by π_{UCT} is the following formula:

$$\pi_{UCT} = argmax\{Q^{"}_{UCT}(s,a) = Q_{UCT}(s,a) + \mathbb{C}\sqrt{\frac{\ln(n(s))}{n(s,a)}}, a \in \mathbb{A}\}, \quad \text{Eq. (1)}$$

Where π_{UCT} is the selected policy. s is system state, a is action, and $\mathbb{A}$ is the set for all actions. $n(s)$ is the total number of times a state s has been visited, and $n(s,a)$is the number of times action a has been selected in state s. $Q_{UCT}(s,a)$ is the empirical cumulative reward, averaged over all iterations, when action a has been selected in state s; C is a problem-dependent parameter to control the balance between exploitation and exploration.

Reward definition

Eq. (2) was defined to calculate the value of reward $Q(s,a)$:

$$Q(s,a) = \frac{\sum_{i=1}^{n(s,a)} X_i}{n(s,a)} \quad \text{Eq. (2)}$$

Where, X_i denotes the reward of ith simulation associated with action a.

Two objective functions were modeled and compared in this manuscript. The first objective focused on the total travel time (TT). The expectation was that by adjusting the movement of CAV, the total TT of the mixed traffic flow can be reduced. As such, the negative value of the total travel time was defined as the reward X_i:

$$X_i = -TT_i = -\sum_{j=1}^{M} \left(t_{f_j} - t_{s_j} \right) \quad \text{Eq. (3)}$$

Where, t_{f_j} denotes the time of jth vehicle (j=1, 2, …, m) passing the stop line, and t_{s_j} is the time of jth vehicle passing the start position.

The second objective focused on the fuel consumption (FC). Here, the VT-micro vehicle fuel consumption and emissions model [25] was applied, as shown in

$$FC_i = \sum_{j=1}^{M} \int_{t_{s_j}}^{t_{f_j}} exp\left(\sum_{u=0}^{3} \sum_{w=0}^{3} K_{uw} v(t)_j^u a(t)_j^w \right) dt \quad \text{Eq. (4)}$$

Where, $v(t)_j$ denotes the velocity of vehicle j (j=1, 2, …, m) at time t, $a(t)_j$ denotes the acceleration of vehicle j (j=1, 2, …, m) at time t. K_{uw} is a constant coefficient from [5], with u and w

With the minimization of fuel consumption as the objective function, the expectation was that by adjusting the movement of CAV, the total FC of the mixed traffic flow can be reduced. As such, the negative value of the total FC was defined as the reward X_i:

$$X_i = -FC_i = -\sum_{j=1}^{M} \int_{t_{s_j}}^{t_{f_j}} exp\left(\sum_{u=0}^{3} \sum_{w=0}^{3} K_{uw} v(t)_j^u a(t)_j^w \right) dt \quad \text{Eq. (5)}$$

State Transition: HDV Car-Following Behavior

While CAV movements were adjusted by the MCTS algorithm, another piece of the moving puzzles was the HDV movement. It is intuitive that when CAV makes a move, the microscopic conditions of the HDV surrounding CAV will change accordingly and the HDV will also make a move. For example, if CAV accelerates, then the HDV that is driving immediately behind CAV will experience a larger headway and may consider accelerating as well. As such, to quantitatively capture the state transition, the HDV's car-following behavior needs to be considered as well.

To illustrate the velocity decision-making of the HDVs, the optimal velocity car-following model in [26] was employed. Such car-following model can be easily replaced with other models such as the non-linear Gazis-Herman-Rothery (GHR) model and others.

$$\ddot{y_i}^{t+t_d} = \alpha\left\{ V\left(d_{h_i}{}^t \right) - \dot{y_i}^t \right\} \quad \text{Eq. (6)}$$

Where, y_i^t denotes the distance from the start position of the ith vehicle at time t. ${d_{h_i}}^t$ denotes the distance headway of the ith vehicle at time t. Coefficient α indicates the sensitivity of the driver. V is a function to determine the optimal velocity with a given ${d_{h_i}}^t$. In this paper parameter α was set to be 2.0 m/sec, and the function V determined by the observed data on Japanese motorways by Sugiyama [27] was employed. The unit of length and time were meter and second, respectively.

$$V\left({d_{h_i}}^t\right) = 16.8\left[tanh 0.0860\left({d_{h_i}}^t - 25\right) + 0.913\right] \quad \text{Eq. (7)}$$

It should be noted that when the HDV is approaching intersections, the driver's response was subject to not only the car-following model but also the traffic signals. If the light is red, the vehicle will start to decelerate when it's within certain distance of the intersection, and then stops in front of the stop line or wait at the end of the queue. If the light is yellow, it will determine if it can pass the intersection before yellow light ends, if not, it will stop and wait for the next green light.

With the basic n-MCTS introduced, next, we start to introduce the two improvements over n-MCTS, i.e. the exploration-exploitation balance calibration module and tree-expansion determination module.

3.2 Exploration-Exploitation Balance Calibration Module

As discussed above, in the n-MCTS algorithm, the constant C can be adjusted to increase or decrease the amount of exploration performed, which also impacted the degree of exploitation and ultimately influenced the solution quality and algorithm convergence. The value C = 1 was shown by Kocsis and Szepesv´ari [28] to satisfy the Hoeffding inequality with rewards in the range [0, 1]. The calibration of C value is needed to improve the modeling quality, or when the reward fell outside of this range.

A two-step exploration-exploitation balance calibration procedure was designed as below. First, based on the lower bound and upper bound of balance parameter c, denoted as l_c and u_c, a step size β is determined by Eq. (8). n_{iter} indicates the numbers of iteration.

$$\beta = (ln(l_c) - ln\,(u_c))/n_{iter} \quad \text{Eq. (8)}$$

Next, the balance parameter c for the nth iteration, denoted as C_n, is calculated via Eq. (9).

$$C^n = exp(ln(l_b) + (n-1)\cdot\beta) \quad \text{Eq. (9)}$$

It should be noted that here the step-size parameter in logarithmic scale was adopted. This is was designed to effectively select parameter values in the case where the upper and lower bounds differ by more than one order of magnitude (e.g. l_c =0.01 whereas u_c=1000).

3.3 Tree-Expansion Determination Module

This section aims to develop rules to more effectively grow the tree along the desired direction. Naïve MCTS algorithm would perform selection–expansion–simulation- and back propagation regardless of the status that CAV is facing. We believe sometimes such operation was neither necessary nor helpful in improving the solution quality. In other words, if the traffic condition was not much changed compared with the last moment, triggering of MCTS does not bring any new information to the simulation, but instead may introduce random noise and grow the tree towards an undesired direction. Additionally, such operation brings significant concerns to the algorithm run time and leads to a waste of memory and CPU resources.

To determine when should the tree expansion be prohibited, we analyze the "marginal impact" of a CAV movement. While CAV performs an action, the HDV that is immediately behind CAV would find a different time headway, and thus its speed might be adjusted according to Eq. (7). To determine the degree of adjustment, we perform the partial derivative and can derive the acceleration/deceleration value as below.

$$\dot{V}\left(d_{h_i}{}^t\right) = \frac{\partial \mathrm{v}}{\partial \mathrm{d}} = 16.8 * \frac{\partial\left[tanh 0.0860\left(d_{h_i}{}^t - 25\right) + 0.913\right]}{\partial \mathrm{d}} \qquad \text{Eq. (10)}$$

$$= 1.448\left(1 - tanh^2 0.0860\left(d_{h_i}{}^t - 25\right)\right) d\left(d_{h_i}{}^t\right)$$

It should be noted that Eq. (10) merely quantify the impact of CAV to the vehicle that follows immediately behind it. If multiple vehicles were following CAV, the impact would propagate to the upstream vehicles in the form of shockwave. As such, the total impact is the summation of all vehicles behind CAV, i.e.

$$\dot{Sum}(V) = \sum \dot{V}\left(d_{h_i}{}^t\right) \qquad \text{Eq. (11)}$$

$$= \sum 1.448 * \left(1 - tanh^2 0.0860\left(d_{h_i}{}^t - 25\right)\right) d\left(d_{h_i}{}^t\right)$$

$\forall i$ behind CAV vehicle.

3.3 MCTS-MTF model

With the exploration-exploitation balance calibration module and tree-expansion determination module, the framework of MCTS-MTF algorithm, which was improved over n-MCTS is shown in Figure 2.

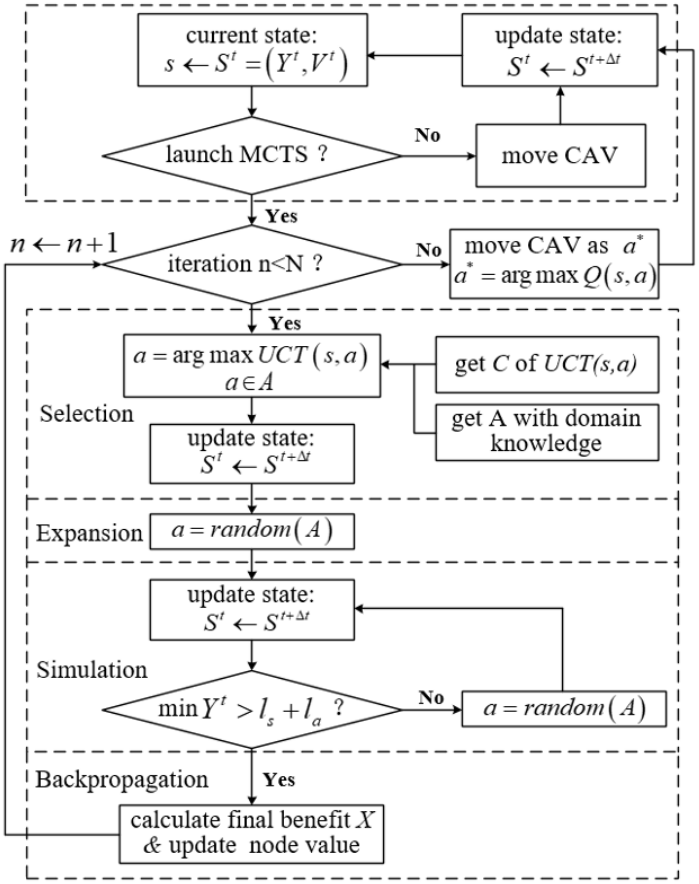


Figure 2. The proposed MCTS-MTF algorithm framework

The model works with the following steps.

1. Start from a current state $S^t = \{Y^t, V^t\}$, in which Y^t is the set of all vehicles' distance from the start position, and V^t is the set of all vehicles' velocity at time t
2. Determine if it's necessary to launch MCTS algorithm via Eq. (10) and Eq. (11). If yes, go to step 4, otherwise go to step 3.
3. Move CAV one step ahead, and update the states of CAV and HDV accordingly. Then go back to step 1.
4. Determine if maximum number of iterations has been reached. If yes, go to step 5, otherwise go to step 6.
5. Update the states of CAV and HDV accordingly, then go back to step 1.
6. Do *Selection*: determine the optimal action for CAV with the UCT function via Eq. (1). It should be noted that the balance parameter C should come from Eq. (9)
7. Update the states of CAV and HDV
8. Do *Expansion*: randomly select a move for CAV to expand the tree
9. Do *Simulation*: update the states of CAV and HDV, and then determine if all vehicles have passed the stop line. If not, go to step 10. Otherwise go to step 11

10. Select the next random move, and go back to step 9.
11. Do *Backpropagation*: calculate the final benefit of X and update the node value. Then go back to step 4.

4 CASE STUDY

In this section, numerical experiment on a one-lane roadway segment was conducted to test the proposed MCTS-MTF algorithm. Considering that the minimum intersection spacing along an arterial corridor was usually set to be a quarter mile, the roadway length was set as 400 meters, i.e. $l_s = 400$ m. Similarly, we empirically chose 70 seconds as the signal cycle (with 30s green time, 5s yellow time and 35s red time), i.e. $t_g = 30$s, $t_y = 5$s, $t_r = 35$s, $t_c = 70$s. Considering the typical congestion on the urban roadway network and the queuing process at intersection, a free flow speed of 8.33m/s (i.e. roughly 20mph) was used. As the lengths of most commonly-seen vehicles range between 3195 mm and 5614 mm, as such we set 5m as the vehicle average length, i.e. $l_a = 5$m. The value of parameter c that balances the MCTS exploration and exploitation was calibrated to be 0.08 through the calibration method described in section 3.2, i.e. $C_p = 0.08$.

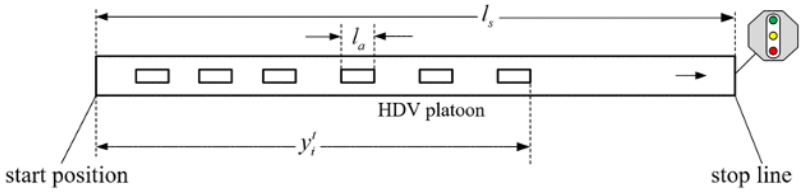


Figure 3. The component of case study benchmark

In the benchmark scenario, six HDVs were simulated as Fig. 3. Then in the n-MCTS and MCTS-MTF models, the first vehicle in the platoon was assigned as the CAV. The objective function was set to be minimizing total travel time (TT), and minimizing total fuel consumption (FC), respectively. After that, the algorithms' performances and running times (RT) were compared.

4.1. Algorithm result analysis

- **Travel time minimization as objective**

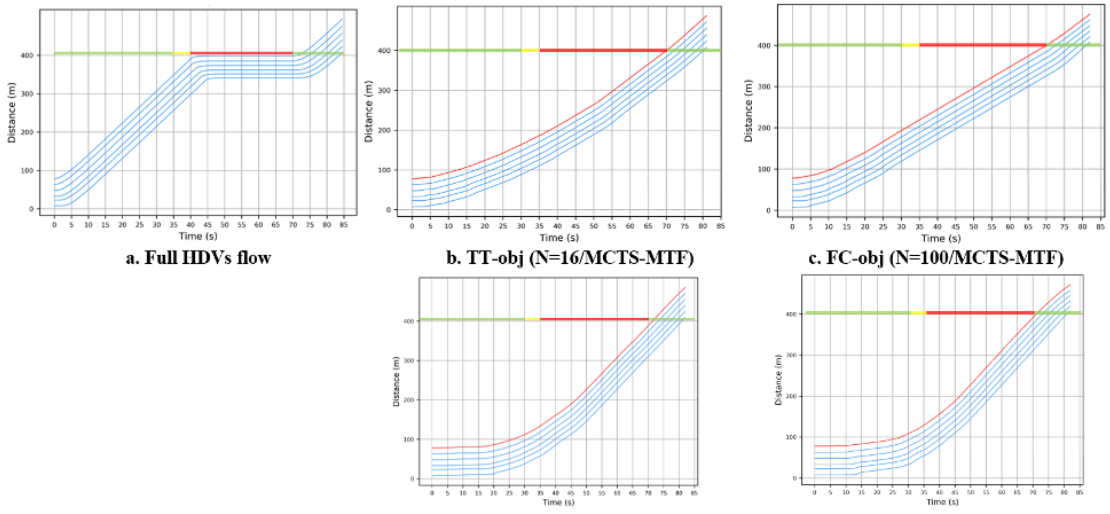
When the total travel time minimization was used as the objective function, the total travel time dropped from 475 seconds in the benchmark scenario to 463.5 seconds with n-MCTS algorithm, and further to 458.5 seconds with MCTS-MTF algorithm. In other words, the travel time reduced by 2.4% with n-MCTS algorithm and by 3.5% with MCTS-MTF algorithm. On the other hand, the total FC dropped from 776.02ml in the benchmark scenario to 747.3ml with n-MCTS algorithm, and further to 736.76ml with MCTS-MTF algorithm. In other words, the FC reduced by 3.7% with n-MCTS algorithm and by 5.1% with MCTS-MTF algorithm.

- **Fuel consumption minimization as objective**

When total fuel consumption minimization was used as the objective function, the total FC dropped from 776.02ml in the benchmark scenario, to 732.57ml with n-MCTS algorithm, and further to 725.38ml with MCTS-MTF algorithm. In other words, the FC reduced by 5.6% with n-MCTS algorithm and by 6.5% with MCTS-MTF algorithm. In both cases the FC consumption were higher than the test above when total TT minimization was used in the objective function. On the other hand, the total travel time dropped from 475 seconds in the benchmark scenario to 462 seconds with n-MCTS algorithm, and further to 461.5 seconds with MCTS-MTF algorithm. In other words, the travel time reduced by 2.7% with n-MCTS algorithm and by 2.8% with MCTS-MTF algorithm.

The vehicles' time-space diagram corresponding to three different algorithms were shown in Figure 4 below. Figure 4-a shows the benchmark scenario, in which we can see the vehicles drove at a constant speed to the intersection, then stopped at the red light and accelerated when the light turned green. Figure 4-b was the results of MCTS-MTF model with TT as the objective, in which the vehicles drove slower than benchmark scenario first, but was able to pass the intersection right after the green light was turned, with a high speed. Similar pattern was observed in Figure 4-d which corresponded to the n-MCTS model with TT as the objective. However, the first vehicle did not pass the intersection until a few seconds after the green light turned on.

Figure 4-c and Figure 4-e corresponded to MCTS-MTF model and n-MCTS model, both with FC as the objective. It can be observed that Figure 4-e exhibit a sharper curve, and thus led to a slightly higher fuel consumption when compared with Figure 4-c. However, in both scenarios the trajectories were much smoother than the benchmark scenario, and thus saved fuel consumptions. From the figure, it can be observed that due to the red signal constraints, the amount of travel time saving is bounded. In contrast, the degree of fuel consumption was largely dependent on the smoothness of the trajectory and thus had more room for improvement.



a. Full IIDVs flow

b. TT-obj (N=16/MCTS-MTF)

c. FC-obj (N=100/MCTS-MTF)

d. TT-obj (N=100/n-MCTS)

e. FC-obj (N=100/n-MCTS)

Figure 4: Time-Space Diagram Comparison

4.2 Algorithm convergence analysis

Figure 5 below shows the changes in fuel consumption with different iteration numbers. The convergence analysis shows when iteration number was set to be 15, the fuel consumption dropped significantly already (by 5%), and as the iteration number increases, the FC continued to drop (by 6.5%, i.e. an additional 1.5%). It was also observed that after 90th iteration, the fuel consumption actually became very stable, the fluctuation was within 5ml, i.e. within 5/730=0.68% and can be considered as converged.

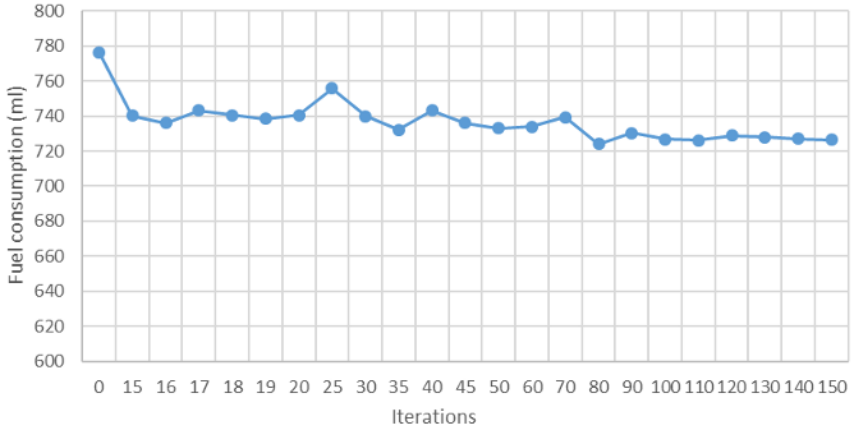


Figure 5. MCTS-MTF convergence analysis - changes of FC

The simulation was done in a computer with Intel(R) Core(TM) i9-9920X 3.50GHz CPU and 64GB memory, the algorithm run time was 73 seconds. Considering the travel time of HDV platoon on this roadway segment was about 82 seconds on average, the algorithm demonstrates good potentials for real-time or near real-time applications. Here, due to the mechanism of MCTS algorithm, if the algorithm run time is less than the total travel time needed for a vehicle to finish traveling on this roadway segment (in other words, the runtime needed to decide one movement is lower than the duration of one time interval), the algorithm would have potential for real-time applications. An analogy of this rational was "AlphaGo" which was developed by Google to play game Go, based on the similar MCTS algorithm. In this process, the entire game may take a few hours, but what's really important is the time required for the computer to make one move. As long as the computer can decide the next movement before the time when decision is needed, the system can move forward without delay.

An example is used to further illustrate this idea in Figure 6 below. As the time interval is set to be 1 second, $\Delta t = 1$. The critical timestamp to examine is time t (i.e. the timestamp by when a decision has to be made), located in the middle of the figure below.

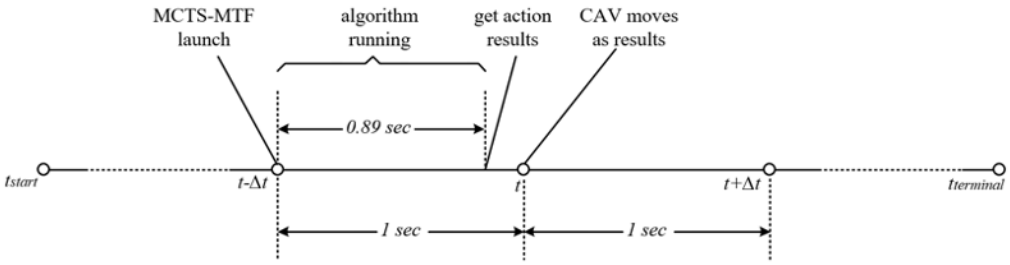


Figure 6: Illustration of MCTS run time requirement

At timestamp t-Δt (i.e. t-1 second), HDV determine their acceleration/deceleration values based on the system status at that time. Once this is determined, their status (e.g. HDV speed, location) in 1 second (i.e. at time t) can be inferred. Timestamp t is the moment that CAV decision needs to be ready. In other words, at time t-Δt, HDV's status at time t can be inferred, and MCTS-MTF is triggered right away to decide the best CAV movement at time t. As long as the MCTS-MTF can finish calculation of the next movement before t (or in other words, within one second), when the HDVs finish simulation at time t, the decision of CAV is already known and thus HDV will be able to gain the complete information on system status, and decide on the next move again. As such, the simulation can continue for one more step, and so on and so forth.

In our case study, the travel time of HDV platoon on this roadway segment was about 82 seconds on average. As such, MCTS-MTF model spends about 73/82=0.89 second to decide the optimal movement policy in one time interval. In other words, in the picture above, the algorithm run time for one-step is 0.89 seconds which is lower than Δt=1 second. As such, when the HDV finishes simulating at the end of each second, the decision of CAV is ready, and the system will continue to evolve continuously without waiting.

4.3 Exploration-Exploitation Balance Parameter Calibration Results

The exploration-exploitation balance parameter C was calibrated from 0.01 to 1 with Eq. (8) and Eq. (9). The results were shown in Figure 7, with the Y-axis in (a) being travel time and in (b) being fuel consumption. The results showed that the objective functions, i.e. TT and FC changed significantly with different C values tested. Interestingly, in these two cases the optimal C values corresponding to the lowest TT and FC were both found to be around 0.08.

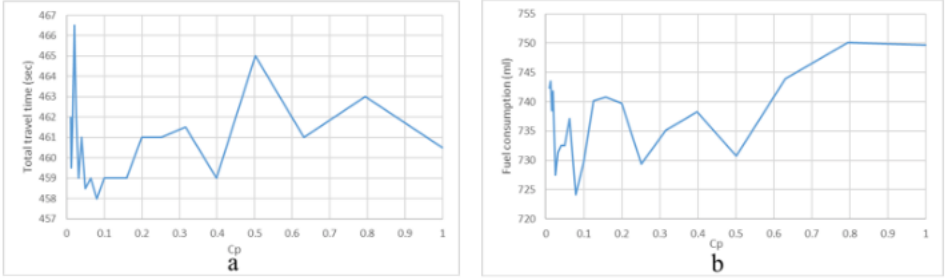


Figure 7. MCTS-MTF balance parameter calibration (a) TT, (b) FC

4.4 Background traffic sensitive analysis

The algorithm's performance in saving TT and FC was further tested with varying level of service (LOS, 1~6 corresponds to A~F), the results were shown in Figure 8 and Table 1. It can be found that for travel time, the maximum saving was observed at LOS D, while minimum saving was observed at LOS A. The guess was that when the traffic was free flowing, not much can be done to save travel time, whereas when the traffic started to get congested, CAV can play a more significant role in smoothing the traffic. In terms of the fuel consumption savings, the most saving was observed when LOS=B, whereas when traffic became congested, the percentage of saving was reduced significantly.

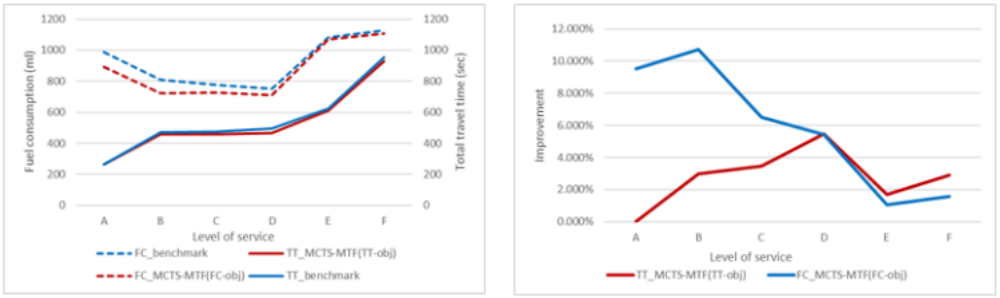


Figure 8. Algorithm performance in different LOS. (a) absolute values, (b) saving percentages

Table 1. MCTS-MTF performance (TT&FC) in different LOS

LOS	TT_benchmark	FC_benchmark	TT_MCTS-MTF	FC_MCTS-MTF
A	264.5	987.02	264.5 (0.000%)	893.25 (9.500%)
B	472	808.7	458 (2.966%)	721.99 (10.722%)
C	475	776.02	458.5 (3.474%)	725.38 (6.526%)
D	493.5	751.82	466.5 (5.471%)	710.91 (5.441%)
E	622	1082.86	611.5 (1.688%)	1071.5 (1.049%)
F	955	1126.29	927.5 (2.880%)	1108.6 (1.571%)

5 CONCLUSION AND FUTURE RESEARCH

Previous researches often simplified the trajectory scheduling problem with certain assumptions to reduce computational burden, such as dividing a vehicle trajectory into several segments with constant speed or linear acceleration/deceleration which was rather unrealistic. This manuscript departed from the existing research that minimum constraints on CAV trajectory control were required, as long as the basic rules such as safety consideration and vehicular performance limit were followed. Modeling efforts were made to improve the algorithm solution quality and the runtime efficiency over naïve MCTS (n-MCTS) algorithm. This was achieved by an exploration-exploitation balance calibration module, and a tree expansion determination

module to more effectively expand the tree along the desired direction. The proposed algorithm was implemented and tested in a one-lane roadway with signalized intersection control. The results found that compared with the benchmark scenario, the proposed algorithm was able to achieve a travel time saving of 3.5% and a fuel consumption of 6.5%. The proposed algorithm was also demonstrated to run 8 times faster than a naïve MCTS model, suggesting a promising potential for real-time or near-real time application.

Future research may be focused on the following topics. First, how can the algorithm work in a general network with multiple lanes is a topic worth investigation, as most existing research can be applied only to a one-lane highway. Further, if on the roadway segment there're multiple CAVs, how can they work collaboratively to achieve a common objective function can also be investigated.

6. AUTHOR CONTRIBUTIONS

The authors confirm contribution to the paper as follows: study conception and design: Y. Cheng, X. Hu, H. Qi; data collection: Y. Cheng, Q. Tang; analysis and interpretation of results: Y. Cheng, H. Qi, H. Yang; draft manuscript preparation: Y. Cheng, Q. Tang, X. Hu. Editing and proofreading: H. Yang. All authors reviewed the results and approved the final version of the manuscript.

7. REFERENCE

1. J. Lee and B. Park, Development and Evaluation of a Cooperative Vehicle Intersection Control Algorithm Under the Connected Vehicles Environment. IEEE Transactions on Intelligent Transportation Systems, 2012. 13(1): p. 81-90.
2. Kaidi Yang, S. Ilgin Guler, and Monica Menendez, Isolated intersection control for various levels of vehicle technology: Conventional, connected, and automated vehicles. Transportation Research Part C: Emerging Technologies, 2016. 72: p. 109-129.
3. X. Wu, X. He, G. Yu, A. Harmandayan, and Y. Wang, Energy-Optimal Speed Control for Electric Vehicles on Signalized Arterials. IEEE Transactions on Intelligent Transportation Systems, 2015. 16(5): p. 2786-2796.
4. M. A. S. Kamal, M. Mukai, J. Murata, and T. Kawabe, Model Predictive Control of Vehicles on Urban Roads for Improved Fuel Economy. IEEE Transactions on Control Systems Technology, 2013. 21(3): p. 831-841.
5. Jiaqi Ma, Xiaopeng Li, Fang Zhou, Jia Hu, and B. Brian Park, Parsimonious shooting heuristic for trajectory design of connected automated traffic part II: Computational issues and optimization. Transportation Research Part B: Methodological, 2017. 95: p. 421-441.
6. Fang Zhou, Xiaopeng Li, and Jiaqi Ma, Parsimonious shooting heuristic for trajectory

design of connected automated traffic part I: Theoretical analysis with generalized time geography. Transportation Research Part B: Methodological, 2017. 95: p. 394-420.

7. Q. Jin, G. Wu, K. Boriboonsomsin, and M. J. Barth, Power-Based Optimal Longitudinal Control for a Connected Eco-Driving System. IEEE Transactions on Intelligent Transportation Systems, 2016. 17(10): p. 2900-2910.

8. Pengfei Li and Xuesong Zhou, Recasting and optimizing intersection automation as a connected-and-automated-vehicle (CAV) scheduling problem: A sequential branch-and-bound search approach in phase-time-traffic hypernetwork. Transportation Research Part B: Methodological, 2017. 105: p. 479-506.

9. Feng Zhu and Satish V. Ukkusuri, A linear programming formulation for autonomous intersection control within a dynamic traffic assignment and connected vehicle environment. Transportation Research Part C: Emerging Technologies, 2015. 55: p. 363-378.

10. Yuguang Wei, Jiangtao Liu, Pengfei Li, and Xuesong Zhou. Longitude trajectory optimization for autonomous vehicles: an approach based on simplified car-following model. in Transportation Research Board 95th Annual Meeting, Washington, DC, USA. 2016.

11. Chunhui Yu, Yiheng Feng, Henry X. Liu, Wanjing Ma, and Xiaoguang Yang, Corridor level cooperative trajectory optimization with connected and automated vehicles. Transportation Research Part C: Emerging Technologies, 2019. 105: p. 405-421.

12. Xiaozheng He, Henry X. Liu, and Xiaobo Liu, Optimal vehicle speed trajectory on a signalized arterial with consideration of queue. Transportation Research Part C: Emerging Technologies, 2015. 61: p. 106-120.

13. Amir Mirheli, Mehrdad Tajalli, Leila Hajibabai, and Ali Hajbabaie, A consensus-based distributed trajectory control in a signal-free intersection. Transportation Research Part C: Emerging Technologies, 2019. 100: p. 161-176.

14. Xiangwang Hu and Jian Sun, Trajectory optimization of connected and autonomous vehicles at a multilane freeway merging area. Transportation Research Part C: Emerging Technologies, 2019. 101: p. 111-125.

15. Xiao Liang, S. Ilgin Guler, and Vikash V. Gayah, Joint Optimization of Signal Phasing and Timing and Vehicle Speed Guidance in a Connected and Autonomous Vehicle Environment. Transportation Research Record, 2019. 2673(4): p. 70-83.

16. Chunhui Yu, Yiheng Feng, Henry X. Liu, Wanjing Ma, and Xiaoguang Yang, Integrated optimization of traffic signals and vehicle trajectories at isolated urban intersections. Transportation Research Part B: Methodological, 2018. 112: p. 89-112.

17. Xiao Liang, S. Ilgin Guler, and Vikash V. Gayah, Signal Timing Optimization with Connected Vehicle Technology: Platooning to Improve Computational Efficiency. Transportation Research Record, 2018. 2672(18): p. 81-92.

18. Yi Guo, Jiaqi Ma, Chenfeng Xiong, Xiaopeng Li, Fang Zhou, and Wei Hao, Joint optimization of vehicle trajectories and intersection controllers with connected automated vehicles: Combined dynamic programming and shooting heuristic approach. Transportation Research Part C: Emerging Technologies, 2019. 98: p. 54-72.

19. Handong Yao, Jianxun Cui, Xiaopeng Li, Yu Wang, and Shi An, A trajectory smoothing method at signalized intersection based on individualized variable speed limits with location optimization. Transportation Research Part D: Transport and Environment, 2018. 62: p. 456-473.

20. Dimitris Bertsimas and Jack Dunn, Optimal classification trees. Machine Learning, 2017. 106(7): p. 1039-1082.

21. Sylvain Gelly and David Silver, Monte-Carlo tree search and rapid action value estimation in computer Go. Artificial Intelligence, 2011. 175(11): p. 1856-1875.

22. Joseph L Nguyen, Nicholas RJ Lawrance, Robert Fitch, and Salah Sukkarieh, Real-time path planning for long-term information gathering with an aerial glider. Autonomous Robots, 2016. 40(6): p. 1017-1039.

23. Jacek Mańdziuk and Maciej Świechowski, UCT in Capacitated Vehicle Routing Problem with traffic jams. Information Sciences, 2017. 406-407: p. 42-56.

24. Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. in European conference on machine learning. 2006. Springer.

25. Kyoungho Ahn, Hesham A. Rakha, and Sangjun Park, Ecodrive Application. Transportation Research Record: Journal of the Transportation Research Board, 2013. 2341(1): p. 1-11.

26. M. Bando, K. Hasebe, A. Nakayama, A. Shibata, and Y. Sugiyama, Dynamical model of traffic congestion and numerical simulation. Physical Review E, 1995. 51(2): p. 1035-1042.

27. Y. Sugiyama, Dynamical model for congestion of freeway traffic and its structural stability. 1996: p. 137.

28. Levente；Szepesvári Kocsis, Csaba；Willemson, Jan. Improved Monte-Carlo Search. 2006.

List of Figure Captions

Figure 1. Decomposition of mixed traffic flow into basic interactive units
Figure 2. The proposed MCTS-MTF algorithm framework
Figure 3. The component of case study benchmark
Figure 4- Time-Space Diagram Comparison
Figure 5. MCTS-MTF convergence analysis - changes of FC
Figure 6- Illustration of MCTS run time requirement
Figure 7. MCTS-MTF balance parameter calibration (a) TT, (b) FC
Figure 8. Algorithm performance in different LOS. (a) absolute values, (b) saving percentages